Kanyon Phillips

CS 454 Midterm

Dr. Schrimpsher

21 October, 2026

CS 454 Midterm Project Report

Resources:

To deploy this to workload on an instructor-provided OpenStack cloud I would choose a Debian server image and m1small for the flavor since this Node.js is stateless with minimal memory usage. It would need a tenant network and a virtual router to connect the internal LAN to external networks and a floating IP. A key pair would be injected during initialization for secure access without password usage. Persistent block storage would be unnecessary for this project as the application's request counter is designed to be temporary and is stored in the local memory of Node.js and resets when the application is shut down. The cloud provider would likely supply the external network pool and everything else would need to be created by me.

The roles of Keystone, Glance, Nova, and Neutron are as follows:

- Keystone: Authenticates credentials and provides a token to allow resource requests.
- Glance: Glance is the VM image registry that supplies the image template to the host server.
- Nova: Allocates the VM to an available hypervisor, schedules the VM, and manages its power state.
- Neutron: Creates internal subnet, sets up router namespace, and assigns fixed internal IP to the VM.

Docker Compose locally operates the Node.js containers within the VM after OpenStack deployment.

Request Path:

1. Client sends an HTTP GET request to the public IP
2. The OpenStack router intercepts the packet and translates the destination from the public IP to the VM's internal IP.
3. Before the packet reaches the VM the OpenStack Security Group inspects it. This requires an explicit ingress rule allowing TCP port 8080.
4. The packet clears security and hits the VM's virtual network interface where Docker's iptables rules intercept the port 8080 traffic and forward it across the internal bridge into the container.
5. The Node.js server processes the request.

Failure analysis:

1. Try to ping the floating IP and check the dashboard to see if the router is functioning and the Security Group ingress rules are properly setup.
2. Attempt to SSH into the instance. If SSH works the VM is online and the network route exists.

3. Run curl http://localhost:8080 if a valid JSON response is returned the application is healthy so it an OpenStack firewall or routing configuration issue.
4. If curl http://localhost:8080 fails Run docker ps to see if the container crashed or docker-compose logs to see if Node.js threw an error and terminated.

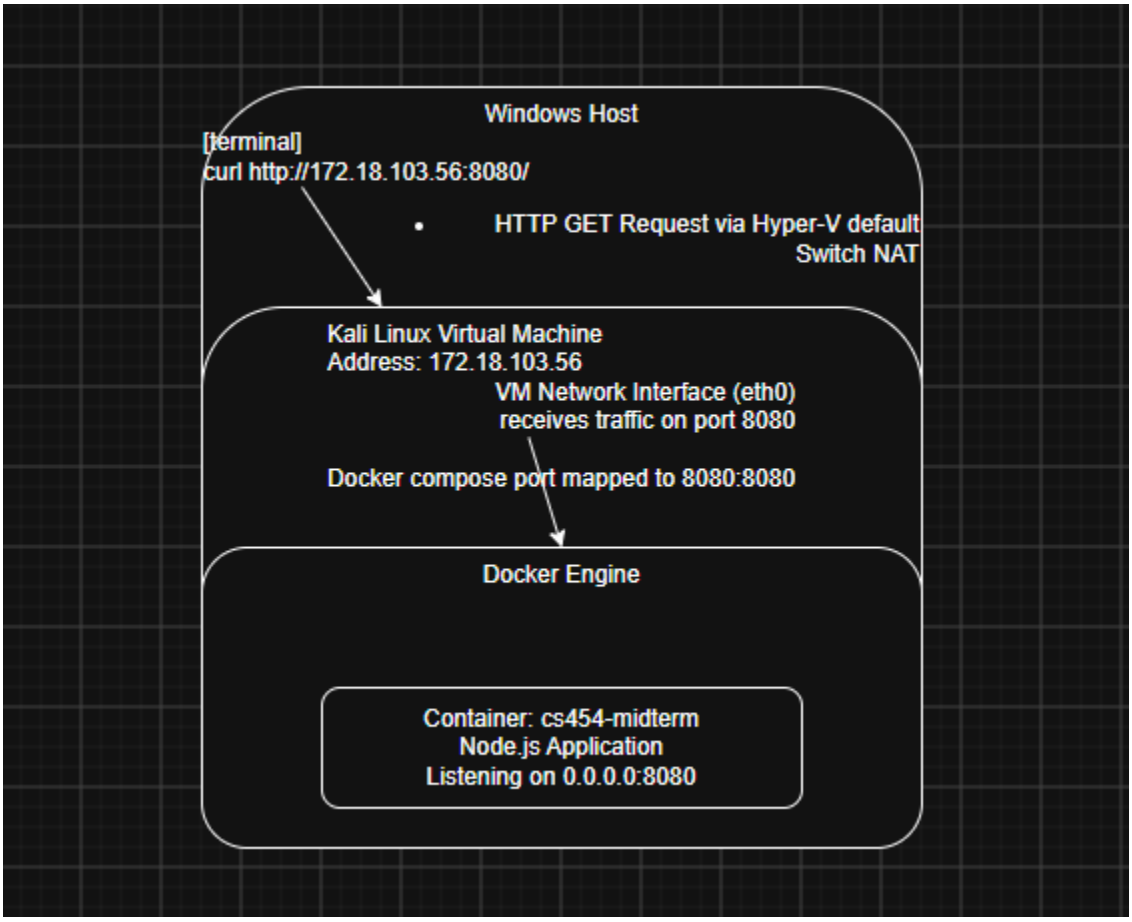
Operational Complexity:

Operating OpenStack involves managing the server, physical storage, hypervisors, etc., that keep the cloud running whereas when deploying to an existing cloud you can already assume that everything is already working so you only have to deal with the software. An infrastructure setup problem would be storage or hypervisors failing while an application deployment problem would be something like an error in compose.yaml or a Dockerfile.

Compose vs Kubernetes:

A VM with compose would be sufficient for simple applications where relogging on crash wouldn't be troublesome. Kubernetes provides greater reliability as shown by the immediately replacement of the deleted pod while it stayed active.

Diagram:



Sources: Lectures and referenced the OpenStack website https://www.openstack.org/software/project-navigator/openstack-components/#openstack-services